

Task 3 — Deterministic UISpec Validator & Tests

Beginner-Friendly Editable Guide

This guide explains what Task 3 built, why it matters, how validation works, how it is tested, and where it fits in the AI-native engineering journey.

1. Where We Were After Task 2

Task 1 established the project engineering foundation. Task 2 converted the written UISpec model contract into a machine-checkable JSON Schema.

```
Written Model Contract
  ↓
JSON Schema
  ↓
Software can check structural validity
```

JSON Schema checks structural rules such as allowed component types, required properties, enums, string lengths, array limits, and component structure. But some rules require additional logic across the complete UISpec. That is the reason for Task 3.

2. What Task 3 Does

Task 3 builds a deterministic UISpec validator in Python and an automated pytest suite. The validator sits between the AI model and the future controlled UI renderer.

```
AI Model
  ↓
Raw UISpec
  ↓
Deterministic Validator
  ↓
Approved UISpec
  ↓
Controlled UI Renderer
  ↓
React UI
```

The validator does not generate UI, call an AI model, render React, or make customer decisions. Its job is to decide whether the UISpec complies with the defined contract.

3. What Does “Deterministic” Mean?

AI models are probabilistic. The same request can sometimes produce different outputs. The validator is different: for the same input, it should produce the same result.

```
Same valid input → VALID
Same valid input → VALID
Same valid input → VALID

Same invalid input → INVALID
Same invalid input → INVALID
Same invalid input → INVALID
```

Key principle: **The validator does not make the AI deterministic. It makes the validation boundary deterministic.**

4. Two Layers of Validation

Layer	Purpose	Examples
JSON Schema	Structural rules	Types, required fields, enums, allowed properties
Runtime validator	Rules needing additional logic	Node count, size, progress relationship, context equality

Think of JSON Schema as checking the shape of the parts, while the validator checks rules about the complete UISpec.

5. Rule #1 — Maximum 30 Nodes

A node means one component instance in the UISpec component tree. Nested child components are also counted.

```
Hero
├── Heading ← node 1
├── Text    ← node 2
└── CTA     ← node 3
```

Hero itself is also a node → total = 4

- One node = one component instance in the UISpec component tree.
- Nested child components count.
- Container components such as Hero, FeatureGrid, and InfoPanel count.
- Their child components count too.
- The UISpec envelope itself is NOT counted.
- JSON properties, metadata fields, strings, and arrays are NOT counted.
- Exactly 30 component instances is valid.
- 31 or more component instances is invalid.

Why? AI-generated UI should remain bounded. The current v0 schema makes the 30-node rule mainly a defensive backstop, but keeping the rule is useful if the component model becomes more expressive later.

6. Rule #2 — Maximum 8 KB

The serialized UISpec must not exceed 8 KiB (8×1024 bytes).

Rule	What it limits
30-node limit	Component-tree complexity
8 KB limit	Serialized data size

The two rules protect different boundaries: one limits structural complexity and the other limits payload size.

7. Rule #3 — ProgressIndicator

If there are three steps, the current position must be less than three.

```
0 = Customer Setup
1 = Document Check
2 = Risk Review
```

```
Valid current values: 0, 1, 2
Invalid current value: 3
```

Rule: **current < len(steps)**

8. Rule #4 — Context Equality

The generated_for information must match the relevant context supplied by the application.

```
Input ContextPayload
segment          = enterprise
product_interest = banking
region           = APAC
```

```
UISpec generated_for
segment          = enterprise
product_interest = banking
region           = Europe
```

→ REJECT

This prevents the generated specification from claiming it was created for a different context than the application actually supplied.

9. Rejection — Not Repair

INVALID MODEL OUTPUT

↓
REJECT

Not:

INVALID → repair → accept

- No truncating text.
- No removing unknown properties.
- No changing invalid CTA actions.
- No fixing invalid progress values.
- No removing extra components.
- No silently changing generated_for.

The validator identifies a problem; it does not hide the problem by changing the model's output.

10. Structured Validation Results

The validator returns a structured result rather than only saying “validation failed.”

```
INVALID
path: components[1].props.action
rule: enum
message: Unsupported CTA action
```

A useful error tells us what failed, where it failed, and why it failed. This helps debugging, testing, observability, and future feedback loops.

11. What Was Implemented

File	Purpose
domain/__init__.py	Python domain package marker.
domain/validator.py	Deterministic UISpec validation and structured results.
tests/domain/test_validator.py	24 automated tests.
pytest.ini	Pytest configuration and Python path setup.
requirements.txt	jsonschema and pytest dependencies.
tests/README.md	Explains deterministic tests and how to run them.

12. The Test Suite

Task 3 created 24 automated tests.

- 5 valid cases: minimal UISpec, Hero + CTA + Text, FeatureGrid, refusal Alert, and ProgressIndicator.
- 12 invalid cases: unsupported components/properties/actions, invalid structure, node limit, size limit, invalid progress value, context mismatch, invalid input, and URL-rule dormancy.
- 2 determinism tests: repeated validation of the same valid and invalid inputs.
- 2 no-repair tests: validator does not mutate the UISpec or context.
- 2 result-shape tests: structured errors are present and do not expose stack traces.

Result: 24 passed in about 0.5 seconds.

13. Task 2 Regression Check

The Task 2 schema verification battery was re-run after Task 3. All 12 checks passed, with no regression.

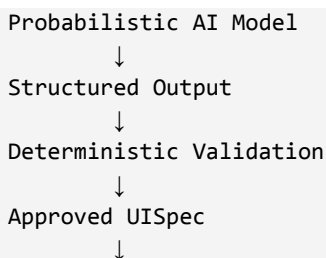
- Draft 2020-12 schema validation
- All 9 v0 components represented
- All approved CTA actions represented
- Invalid examples rejected
- Valid examples accepted
- FeatureGrid rules verified
- Hero headline limit verified

14. What Task 3 Did NOT Build

- React application
- FastAPI application
- AI model integration
- Model gateway
- Authentication
- Document processing
- Customer registration
- Risk AI
- Human approval workflow
- Production infrastructure

This is intentional. The project is being built incrementally, with each task establishing one controlled piece before the next.

15. Where Task 3 Fits



Controlled Components



Rendered UI

Task 1 → Project Engineering Foundation



Task 2 → Machine-Checkable Model Contract



Task 3 → Deterministic Validation Boundary



Future → Controlled Generative UI Renderer

This is the beginning of a controlled model-driven architecture: AI can propose, but deterministic software controls what is allowed to proceed.

16. Why This Matters for AI-Native Engineering

Traditional software is mostly deterministic. AI-native software introduces probabilistic components. Strong boundaries are therefore needed around those components.

AI proposes



Contract defines what is allowed



Schema checks structure



Validator checks runtime rules



Renderer controls what can actually appear

17. Git Checkpoint

Task 3 implementation was checkpointed after verification.

31d4674 Task 3: Implement UISpec validator and tests

The walkthrough documentation was maintained separately:

6ac1e46 docs: extend project walkthrough

These checkpoints create known-good, recoverable project states.

18. Task 3 in One Sentence

Task 3 creates the deterministic gate that decides whether an AI-generated UISpec is allowed to continue toward the UI renderer.

19. Task 2 vs Task 3

Task 2 asks:
"Is the structure correct?"

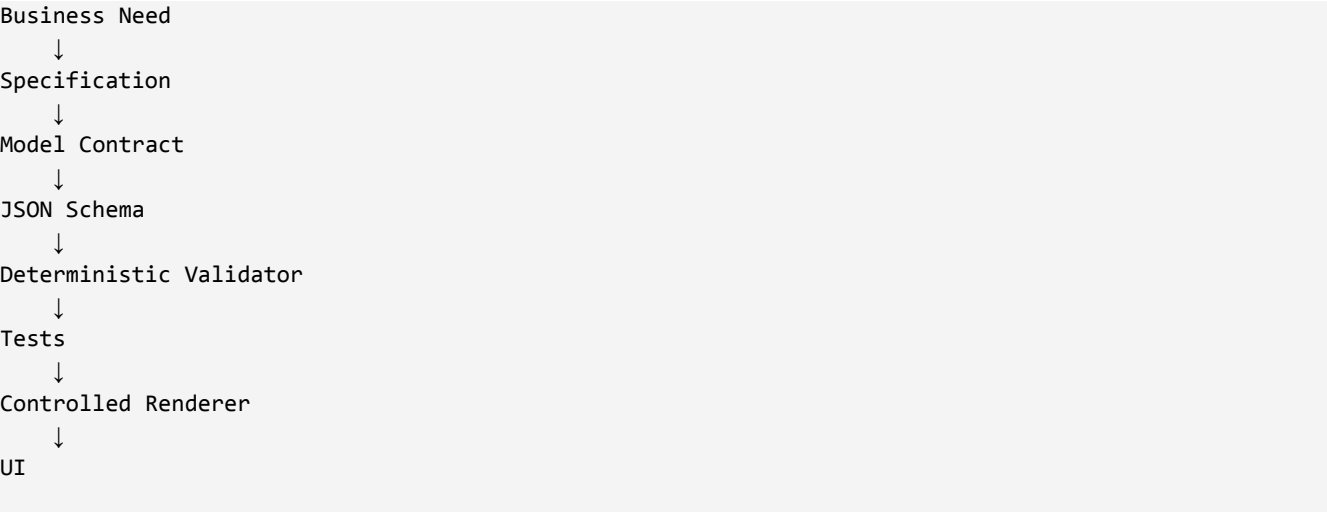
Task 3 asks:
"Does the complete UISpec obey the runtime rules?"

Together, they create a stronger boundary around probabilistic AI output without pretending that the AI itself is deterministic.

20. Current Project State

Checkpoint	Status
Task 1 — Project foundation	Completed and checkpointed
Task 2 — UISpec JSON Schema	Completed and checkpointed
Task 3 — Validator + tests	Completed, verified, and checkpointed
Task 4 — Next implementation increment	Not started

21. Beginner Takeaway



We are deliberately building a control boundary around AI before allowing AI-generated specifications to reach a real UI. This incremental approach is the foundation for the later Customer Onboarding AI application.

Task 3 complete — ready for review before the next implementation increment.

Documentation Clarification — Where the Python Validator Lives

The Task 3 implementation introduced a Python validator. The beginner guide explains what the validator does, but it is useful to explicitly identify the actual files created in the repository.

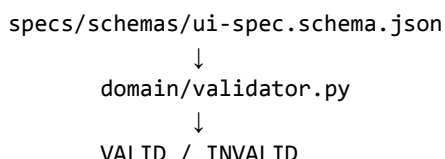
The Python validator

domain/validator.py

This Python module is the executable implementation of the deterministic UISpec validation boundary. It uses the machine-checkable JSON Schema created in Task 2 and applies the additional runtime rules defined in Task 3.

The JSON Schema and Python validator work together

The JSON Schema defines the structural contract. The Python validator applies that contract and also checks runtime relationships and limits that are not conveniently expressed by JSON Schema alone.



The validator's tests

tests/domain/test_validator.py

This test suite verifies that the Python validator behaves deterministically and correctly for valid and invalid UISpecs.

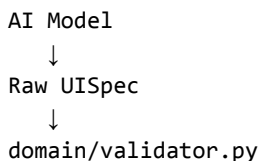
What the Python validator does not do

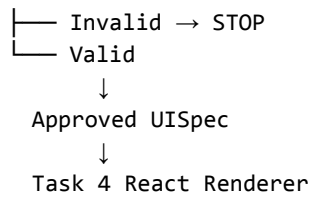
- It does not call the AI model.
- It does not generate UI.
- It does not render React components.
- It does not replace the JSON Schema.
- It does not automatically repair invalid model output.

How Task 2, Task 3 and Task 4 connect

Task	Actual artifact	Purpose
Task 2	specs/schemas/ui-spec.schema.json	Define the machine-checkable UISpec structure.
Task 3	domain/validator.py	Deterministically validate the UISpec.
Task 3	tests/domain/test_validator.py	Verify validator behavior.
Task 4	ui/src/registry.ts	Map approved component names to React components.
Task 4	ui/src/render.tsx	Render the approved UISpec.

The conceptual flow





Important architectural clarification: the Task 3 Python validator and the Task 4 TypeScript/React renderer are not yet physically connected through a runtime API. Task 3 established the deterministic validation boundary; later runtime work will establish the actual connection between the Domain Runtime and UI Runtime.

Complete Task 3 Record — Prompt, Implementation and Checkpoint Context

This section consolidates the actual Task 3 Claude Code prompt used for the project and the implementation details that became important during review before Task 4. It is included so a beginner can trace the work from the instruction given to Claude Code, to the files created, to the validation results and architectural meaning.

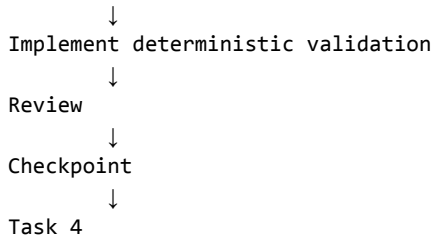
1. Where Task 3 Started

Task 3 started from a verified Task 2 checkpoint. The Task 2 machine-checkable UISpec schema had already been created and the repository was synchronized with the remote repository.

Task 2 checkpoint

3824b07 Task 2: Formalise UISpec model contract

Task 3 starts here



2. Exact Claude Code Prompt Used for Task 3

The following is the Task 3 implementation prompt used in the project. It is the source instruction for creating the deterministic validator and its tests.

We are now starting Task 3 of the Customer Onboarding AI project.

The repository is currently at a verified and remote-synchronized Task 2 checkpoint.

IMPORTANT:

Do NOT start Task 4.

This task is ONLY about implementing the deterministic UISpec validation layer and its tests.

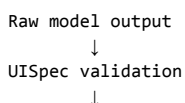
Do not build the React application.
Do not build the FastAPI application.
Do not integrate an AI model.
Do not create a model gateway.
Do not create authentication.
Do not create document processing.
Do not create customer registration.
Do not create risk AI.
Do not create approval workflows.
Do not create production infrastructure.
Do not redesign the existing model contract.
Do not add capabilities that are not required by the current specification.

TASK 3 OBJECTIVE

Build a deterministic validator for the UISpec model contract.

The validator will eventually sit between the probabilistic AI model output and the controlled UI renderer.

The conceptual boundary is:



Approved UISpec



Controlled renderer

The validator must make a deterministic PASS/REJECT decision.

1. READ THE EXISTING PROJECT FIRST

Before creating or modifying anything, read:

- CLAUDE.md
- README.md
- docs/business-problem.md
- docs/architecture.md
- specs/spec.md
- specs/model-contracts.md
- specs/schemas/ui-spec.schema.json
- specs/schemas/README.md
- tests/README.md
- evals/README.md

Do not assume the contents.

Use the existing documents as the source of truth.

2. INSPECT THE CURRENT GIT STATE

Confirm:

- current branch
- current commit
- working tree status
- current repository structure

The repository should start from the Task 2 checkpoint:

3824b07 Task 2: Formalise UISpec model contract

Do not modify anything until you have inspected the current state.

3. DEFINE THE VALIDATOR RESPONSIBILITY

The validator must be deterministic.

For the same input, it must always produce the same validation result.

The validator must NOT:

- call an AI model
- make network calls
- access a database
- execute arbitrary code
- modify the UISpec
- "repair" invalid model output
- invent missing fields
- silently change values
- make business decisions
- approve customers
- render UI

The validator should answer:

"Does this UISpec comply with the defined contract?"

4. JSON SCHEMA VALIDATION

Use:

specs/schemas/ui-spec.schema.json

as the structural source of truth.

The validator should use an appropriate JSON Schema validation library.

Do not rewrite the schema into application code.

The schema should remain the authoritative structural contract.

The validator should report useful validation failures rather than only returning a generic failure message.

----- 5. IMPLEMENT THE RULES DELEGATED FROM TASK 2 -----

The validator must implement the constraints that JSON Schema could not fully express.

These were explicitly documented in:

specs/schemas/README.md

At minimum, implement:

A. TOTAL NODE COUNT

The total number of UISpec nodes across the component tree must not exceed 30.

B. SERIALIZED UISPEC SIZE

The serialized UISpec must not exceed 8 KB.

C. PROGRESS INDICATOR RELATIONSHIP

For ProgressIndicator:

current must satisfy:

current < len(steps)

The validator must reject invalid values.

D. GENERATED CONTEXT EQUALITY

The values in:

meta.generated_for

must match the corresponding values from the input ContextPayload.

Do not merely check that the values are valid enums.

They must match the supplied context.

----- 6. URL HOST ALLOW-LIST -----

Review the existing contract and schema documentation carefully.

If the current v0 contract contains no URL-bearing component properties, do NOT invent URL validation functionality.

Document that the URL host allow-list remains dormant/not applicable for the current v0 component vocabulary.

Do not introduce URL fields merely to implement this rule.

----- 7. VALIDATION RESULT -----

Define a clear deterministic validation result.

For example, conceptually:

VALID

or

INVALID

with structured error information.

The exact shape should be derived from the existing project conventions rather than invented unnecessarily.

A useful validation failure should identify:

- what failed
- where it failed
- why it failed

For example:

component[2].props.headline

or:

component[1].children[3]

Avoid leaking implementation-specific stack traces as the primary validation result.

8. DO NOT AUTO-REPAIR

The validator must reject invalid output.

It must NOT:

- truncate strings
- remove unknown properties
- replace invalid component types
- change CTA actions
- fix `ProgressIndicator.current`
- remove excess nodes
- alter `generated_for`
- silently make the model output valid

The principle is:

INVALID → REJECT

not:

INVALID → REPAIR → ACCEPT

9. TESTS

Create deterministic automated tests for the validator.

The tests must cover at least:

VALID CASES

1. Valid minimal UISpec
2. Valid Hero + CTA + Text
3. Valid FeatureGrid with valid FeatureCard children
4. Valid refusal Alert
5. Valid ProgressIndicator

INVALID CASES

6. Unknown component type
7. Unknown component property
8. Invalid CTA action

9. Invalid component structure
10. More than 30 total nodes
11. UISpec larger than 8 KB
12. ProgressIndicator.current >= len(steps)
13. meta.generated_for does not match ContextPayload
14. Invalid schema structure
15. Attempted unsupported URL-bearing structure if applicable to the contract

For every invalid test, verify that the validator rejects the input.

For every valid test, verify that the validator accepts the input.

10. TEST DETERMINISM

Include at least one test demonstrating that the same input produces the same validation result when validated repeatedly.

The validator must not contain randomness or model-dependent behavior.

11. KEEP TESTS CLOSE TO THE CONTRACT

Do not create a large testing framework.

Keep the test suite small and understandable.

Tests should clearly demonstrate:

Contract rule
↓
Input
↓
Expected result

Prefer readable fixtures/examples over unnecessary abstraction.

12. DEPENDENCY DISCIPLINE

Before adding a dependency:

1. Check whether an existing dependency already supports JSON Schema validation.
2. If no suitable dependency exists, choose one small, well-established JSON Schema validation library appropriate for the implementation language.
3. Do not add unrelated dependencies.

Do not introduce React, FastAPI, model SDKs, database libraries, or infrastructure dependencies.

13. DOCUMENTATION

Update the relevant test/validator documentation only as necessary to explain:

- what the validator does
- what JSON Schema handles
- what runtime validation handles
- what the validator intentionally does NOT do

Do not rewrite the existing model contract.

14. VERIFY THE IMPLEMENTATION

After implementation:

Run the complete validator test suite.

Also verify:

- all existing Task 2 schema checks still pass
- valid examples pass
- invalid examples fail
- no test relies on an AI model
- no network access is required
- validator results are deterministic

15. REVIEW THE DIFF

Before finishing:

Run:

git status

git diff --stat

git diff

Review all changes.

Confirm that the implementation is limited to Task 3.

Do not modify unrelated files.

16. IMPORTANT STOPPING RULE

Do NOT create a Git commit yet.

Do NOT push anything.

At the end of Task 3 implementation, stop and report:

1. What was implemented
2. Files created
3. Files modified
4. Dependencies added, if any
5. Validation rules implemented
6. Tests created
7. Test results
8. Existing Task 2 checks status
9. Any ambiguities discovered
10. Current git status
11. Proposed next checkpoint

Do not start Task 4.

Do not commit the changes.

We will review the Task 3 implementation before creating the next checkpoint.

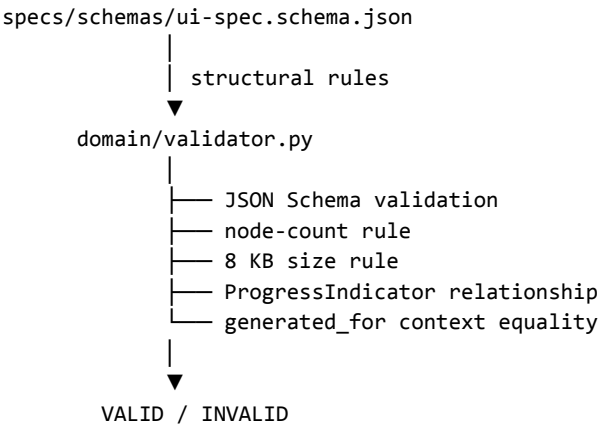
3. What That Prompt Actually Caused Claude Code to Build

The prompt did not explicitly dictate every filename, but it clearly required a Python deterministic validator and a pytest suite. Claude Code implemented that requirement with the following artifacts:

Artifact	Purpose	Role
domain/validator.py	Executable deterministic validation logic	Task 3 validator
domain/_init_.py	Python package marker	Supports the domain module
tests/domain/test_validator.py	Automated validator tests	Task 3 proof
pytest.ini	Pytest configuration	Test execution
requirements.txt	Python dependencies	Includes JSON Schema / pytest dependencies
tests/README.md	Test documentation	Explains deterministic test scope

4. The Most Important File: domain/validator.py

The phrase 'Python validation' refers specifically to the implementation in domain/validator.py. This file is the deterministic gatekeeper. It does not call the AI, render React, or make customer decisions.



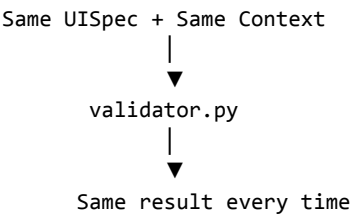
5. JSON Schema vs Python Validator

The two are deliberately different. The JSON Schema remains the structural source of truth; Python implements the complete deterministic validation boundary, including cross-field and runtime rules that JSON Schema cannot conveniently express.

Layer	Main question	Example
JSON Schema	Is the UISpec structurally shaped according to the contract?	Is CTA action one of the allowed enum values?
Python validator	Does the complete UISpec obey runtime/cross-field rules?	Is ProgressIndicator.current < len(steps)?

6. What 'Deterministic' Means Here

Deterministic means that the same UISpec and relevant ContextPayload produce the same validation result. The validator does not use randomness, current time, network state, an AI model, or hidden external state.



7. The 30-Node Rule — Clarified

A node is one component instance in the UISpec component tree, including nested children. The outer UISpec envelope is not a node.



Total = 4 nodes

JSON properties, metadata fields, strings, and arrays are not nodes. The rule is exactly: 30 component instances are allowed; 31 or more are rejected.

During implementation review, we observed that the current v0 schema makes the 30-node limit difficult to reach because the currently permitted container structures are shallow. It remains a useful defensive boundary for future schema evolution.

8. The 8 KB Rule — Clarified

Important: the 8 KB value is the defined v0 project constraint. It is not a universal Generative UI rule. It provides a simple, explicit defensive boundary for this first implementation.

This gives the project two complementary validation layers:

JSON Schema → structural validity

Deterministic validator → runtime and cross-field constraints

The Task 3 validator therefore checks the 30-node limit, the 8 KB size limit, the `ProgressIndicator.current < len(steps)` relationship, and `meta.generated_for` equality.

The 8 KB constraint was deliberately delegated to the deterministic validator because JSON Schema does not fully express this operational serialized-size check. Task 2 established the machine-checkable structural contract; Task 3 adds runtime checks that JSON Schema could not fully express.

Relationship to Task 2 and Task 3

So the two checks are independent:

30 nodes + 7 KB → PASS

30 nodes + 9 KB → REJECT because of size

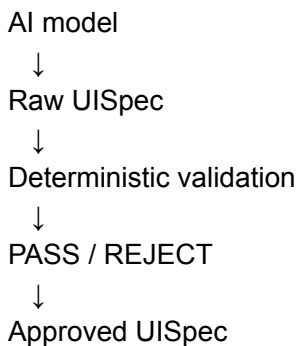
31 nodes + 7 KB → REJECT because of node count

A UISpec must satisfy both conditions.

The rule applies to the serialized JSON representation rather than the in-memory Python object. Conceptually, the validator serializes the UISpec and measures the resulting UTF-8 byte length. If that serialized representation is greater than 8,192 bytes, the validator rejects it.

How the size is measured

Conceptually:



The runtime AI model is probabilistic, so the engineering system should not rely on the assumption that the model will always generate a small UI specification. The 8 KB rule provides a deterministic boundary around model output. Regardless of what the model proposes, the validator accepts only a UISpec whose serialized representation is within the defined limit.

Why this matters in an AI-native system

A UISpec is accepted only when it satisfies BOTH limits: no more than 30 component nodes AND no more than 8 KB of serialized JSON.

Node count and payload size measure different things. A UISpec could contain relatively few nodes but still carry a large amount of text or other allowed data. Conversely, a UISpec could contain many small nodes and remain relatively small. The validator therefore checks both constraints independently.

Why both rules are needed

20 nodes + 5 KB serialized UISpec → PASS

30 nodes + 7 KB serialized UISpec → PASS

30 nodes + 8 KB serialized UISpec → PASS

30 nodes + 9 KB serialized UISpec → REJECT

31 nodes + 5 KB serialized UISpec → REJECT

31 nodes + 9 KB serialized UISpec → REJECT

Examples

The correct interpretation is that all nodes and all of their associated data together must fit within the 8 KB limit. It does NOT mean that each node may contain up to 8 KB.

Therefore, the 30-node limit controls structural complexity, while the 8 KB limit controls total payload size.

The project has two separate deterministic constraints:

- Maximum total component nodes = 30
- Maximum serialized UISpec size = 8 KB (8,192 bytes)

What the 8 KB rule means together with the 30-node rule

This is a limit on the entire UISpec — not a limit of 8 KB per component or per node. The serialized size includes the component tree, component properties, metadata, strings, arrays, and the other JSON content that makes up the UISpec.

In this project, 8 KB means $8 \times 1024 = 8,192$ bytes. The limit applies to the complete serialized JSON representation of the UISpec.

The serialized UISpec must not exceed 8 KB.

9. Context Equality

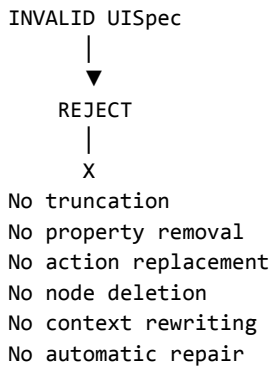
The validator does more than check that `generated_for` values are valid enums. It compares the relevant values in `meta.generated_for` with the supplied `ContextPayload`. A mismatch is rejected.

The implementation intentionally compares only the context fields that the current UISpec contract requires the model to echo. Fields present in `ContextPayload` but not required in `meta.generated_for` are not silently added to the contract.

10. URL Allow-List: Why It Was Not Implemented

The Task 2 documentation identified URL host allow-listing as a future/dormant rule. The current v0 component vocabulary does not contain URL-bearing component properties, so Task 3 did not invent URL fields or URL validation.

11. Validation Means Reject — Not Repair



This is important because automatic repair could hide model failures. The validator's job is to create a clear deterministic boundary.

12. Task 3 Test Result

The implemented pytest suite contained 24 tests covering valid examples, invalid contract cases, runtime constraints, determinism, non-mutation, and validation-result structure.

Task 3 validator tests
24 passed

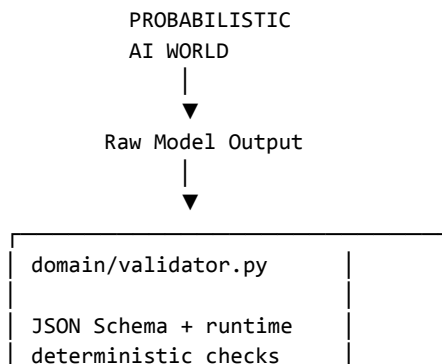
Task 2 schema verification
12 / 12 checks passed

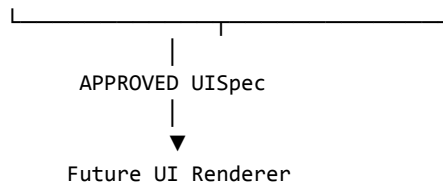
Result: no regression

13. Implementation Observations Discovered During Review

- The current v0 schema makes the 30-node limit difficult to reach; the validator still implements it as a defensive backstop.
- The 8 KB limit is similarly difficult to reach under current field limits but remains a useful explicit boundary.
- The validator does not auto-repair invalid UISpecs.
- The URL host allow-list remains dormant because v0 has no URL-bearing properties.
- The generated_for equality check compares the fields required by the current contract rather than inventing additional echo requirements.
- A namespace collision caused by tests/domain/__init__.py was identified and removed during implementation so pytest collection remained clean.
- Python __pycache__ directories are test byproducts and were intentionally not committed.

14. Architecture at the End of Task 3





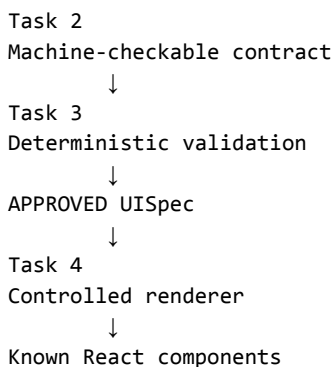
Important: this is an architectural boundary, not yet a physical runtime connection between Python and React. The actual Domain Runtime/API connection will be established later.

15. What Task 3 Did NOT Build

- No React application.
- No controlled React renderer.
- No FastAPI application.
- No runtime model integration.
- No model gateway.
- No authentication.
- No document processing.
- No customer registration.
- No risk AI.
- No approval workflow.
- No production infrastructure.

16. Why Task 4 Can Now Begin

Task 3 established the deterministic gate. Task 4 can therefore focus on the next responsibility: turning an already approved UISpec into known React components through a controlled registry and renderer.



The renderer must receive an approved UISpec rather than raw model output. This keeps validation and rendering as separate responsibilities.

17. Pre-Task-4 Checkpoint Status

Before Task 4, the project should be treated as a known-good Task 3 state only after the implementation has been reviewed and checkpointed in Git. The Task 4 implementation report should not be confused with the Task 3 checkpoint.

The Task 3 implementation itself was reviewed with the following evidence: 24 validator tests passed, all 12 Task 2 schema verification checks passed, no AI/runtime integration was added, and the implementation remained within Task 3 scope.

18. Beginner Takeaway

Task 3 is where the project turns the written UISpec contract into executable deterministic validation. The JSON Schema defines structural validity; domain/validator.py applies the complete validation boundary; the tests prove that boundary behaves consistently; and only an approved UISpec is allowed to move toward the future renderer.